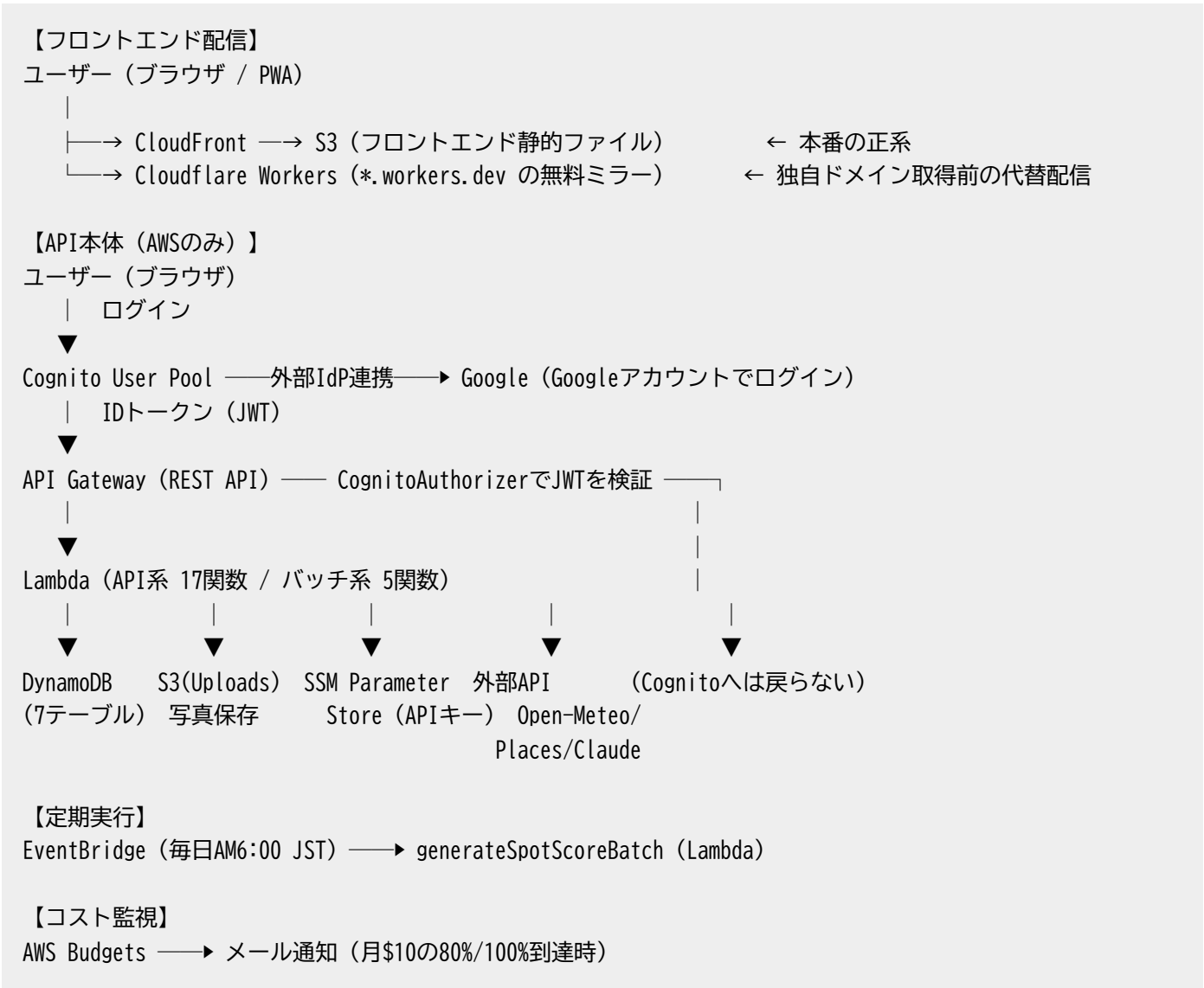


AWS構成・サービス連携ガイド

このアプリで実際に使っているAWS（およびAWS外の関連）サービスを一覧化し、「何のために」「なぜそれを選んだか」「他のどのサービスと連携しているか」をサービス単位でまとめたもの。Lambda関数単位の詳細は[詳細設計.html](#)、ファイル単位の対応は[ファイル構成・依存関係.html](#)を参照。本資料は2026-07-26時点（Gemini→Claude Haiku移行後）の構成を反映している。

1. 全体構成図



2. サービス一覧（早見表）

サービス	役割	主な連携先
Amazon Cognito	Googleアカウントでのログイン認証	Google（外部IdP）、API Gateway（JWT検証）
Amazon API Gateway	フロントからのHTTPリクエストの受付・認証・Lambdaへのルーティング	Cognito、Lambda（22関数）
AWS Lambda	実際の処理を行うプログラム本体（API系・バッチ系の2パッケージ）	DynamoDB、S3、SSM、外部API、他のLambda（非同期起動）
Amazon DynamoDB	アプリの全データ（スポット・おすすめ・投稿・チャット等）の保存先	Lambda
Amazon S3	①フロントエンドの静的ファイル配信 ②ユーザーがアップロードする写真の保存	CloudFront（①）、Lambda・ブラウザ直PUT（②）
Amazon CloudFront	フロントエンドをCDN経由で高速配信（本番の正系）	S3（オリジン）
Cloudflare Workers（AWS外）	フロントエンドの無料ミラー配信（独自ドメイン未取得のため）	フロントのビルド成果物を直接デプロイ（AWSとは独立）
AWS Systems Manager Parameter Store（SSM）	外部APIキー（Claude・Google Places）の安全な保管	Lambda（実行時に取得）
Amazon EventBridge	毎日決まった時刻にバッチLambdaを自動起動するスケジューラ	generateSpotScoreBatch（Lambda）
AWS Budgets	AWS利用料が閾値を超えたらメール通知	Eメール（rfunao0955@gmail.com）
AWS IAM	各Lambdaに必要最小限の権限だけを付与	全AWSサービスへのアクセス制御
GitHub Actions（AWS外）	mainブランチへのpushをトリガーに自動デプロイ	AWS（SAM CLI経由）、Cloudflare、S3、CloudFront

3. 各サービスの詳細

■ Amazon Cognito (User Pool + Google連携)

ログイン機能そのもの。「Googleアカウントでログイン」ボタンを押すとCognitoのHosted UIに飛び、Cognitoが裏でGoogleと認可コードのやり取りをして、最終的にアプリ向けのIDトークン（JWT）を発行する。このJWTがAPI Gatewayでの認証に使われる。

なぜCognito: 自前でパスワード管理・OAuthフローを実装する必要が無く、Googleログインの実装をCognitoに任せられる。API GatewayのCognitoAuthorizerと標準で統合できる点も採用理由。

■ Amazon API Gateway (REST API)

フロントエンドが呼び出す全エンドポイント（/recommendations、/chat、/posts等）の窓口。ログイン必須のエンドポイントにはCognitoAuthorizerをアタッチしており、JWTが無効・未提示なら該当Lambdaを呼ばずに401を返す。

なぜAPI Gateway: Lambdaを直接インターネットに公開せず、認証・CORS・ルーティングを一括管理できる。統合タイムアウトが29秒固定という制約があり、これがAIチャット（同期呼び出し、Lambda側25秒に短縮）とAI分析（非同期呼び出しに変更）の設計判断に直接影響している（詳細設計.html参照）。

■ AWS Lambda (API系・バッチ系の2パッケージ)

実処理の本体。backend/lambda/api/（17関数、handlers.py）と backend/lambda/batch/（5関数、generate_score.py等）の2つのSAM CodeUriパッケージに分かれており、この2つの間でコードは共有できない（batch_common.pyのようなヘルパーが両パッケージに重複して存在する理由）。

なぜLambda (サーバーレス): 個人利用規模のトラフィックでは「常時起動のサーバー」は過剰投資になる。呼ばれた時だけ課金される従量課金と、インフラ管理不要な点を重視した。

なぜAPI系とバッチ系を分けたか: API系はユーザー操作に対して即座に応答する軽量の処理（DynamoDBの読み書き中心）、バッチ系はGoogle Places APIや Claude APIを呼ぶ時間のかかる処理、という性質の違いがあるため。バッチ系は adminRunAiBatchのように「本体を非同期起動するだけの薄いLambda」を挟む構成にして、API Gatewayの29秒タイムアウトを回避している。

■ Amazon DynamoDB (7テーブル)

テーブル名	役割
-------	----

fishing-spots	釣り場マスタ（座標・名前・魚種・写真URL等）
fishing-recommendations	スポットごとの最新スコア・AIコメント（1スポット1件、上書き型）
fishing-favorites	お気に入り登録
fishing-posts	釣果投稿（本文・写真・釣れた魚種）
fishing-chats	AI相談チャットの会話履歴
fishing-usage	1日あたりのAI/API呼び出し回数（レート制限用、TTLで自動削除）
fishing-users	ユーザーの表示名（ユーザー名）

なぜDynamoDB: スキーマがシンプルでアクセスパターンも単純（PK検索・全件scan中心）なため、RDBのような複雑なクエリは不要。PAY_PER_REQUESTモードで使った分だけ課金され、サーバーレスLambdaとの相性も良い。

■ Amazon S3（2バケット）

①フロントエンド配信用バケット：Reactのビルド成果物（HTML/JS/CSS）を置き、CloudFrontがここを参照する。②UploadsBucket：ユーザーが投稿・スポットに設定する写真の保存先。フロントはLambda（presignUploadHandler）から発行される署名付きPOSTフォームを使ってブラウザから直接S3へアップロードし、Lambda自体は画像バイナリを経由しない（API Gatewayのペイロードサイズ制限回避のため）。

なぜ署名付きアップロード: 画像バイナリをLambda経由にするとAPI Gatewayのペイロード上限に引っかかる。S3への直接アップロードにすることでこの制約を回避しつつ、content-length-range条件でアップロード上限（8MB）をS3側に強制させている。

■ Amazon CloudFront + Cloudflare Workers（フロント配信が2系統ある理由）

本番の正系はCloudFront（S3をオリジンとするCDN）。index.htmlはno-cache、ハッシュ付きアセット（/assets/index-<hash>.js）は長期キャッシュという構成。加えて、独自ドメインを取得する前の段階で見目の良いURLを用意するため、Cloudflare Workers（*.workers.devの無料サブドメイン）にも同じビルド成果物を並行デプロイしている。バックエンドAPI（API Gateway）はAWS側1本のみで共有しており、フロントの配信経路だけが2系統ある形。

■ AWS Systems Manager Parameter Store（SSM）

Claude APIキー（/fishing-ai/anthropic-api-key）とGoogle Places APIキー（/fishing-ai/google-places-api-key）をSecureStringとして保管。Lambdaの環境変数に直書きせず、実行時にssm:GetParameterで取得する。

なぜSSM: 環境変数への直書きよりも安全（IAMで読み取り権限を 個別Lambdaごとに絞れる、CloudFormationのコード上に平文キーが残らない）。AWS Secrets Managerより低コストな点も理由（Standard利用なら無料）。

■ Amazon EventBridge

cron(0 21 * * ? *) (UTC 21:00 = 日本時間AM6:00) で generateSpotScoreBatchを毎日自動起動し、全スポットのスコアを 最新化する。ユーザーが「AI分析を実行」 ボタンを押さなくても、朝には最新の おすすめが用意されている状態を作るための仕組み。

■ AWS Budgets

月\$10のAWS利用料予算を設定し、80%/100%到達時にメールで通知する（template.yamlのCostBudgetリソース）。ただしこれはAWS側の課金のみを見ており、Google Cloud（Places/Maps）や Anthropic（Claude）の課金はAWSの外で発生するため対象外。それぞれGoogle Cloud ConsoleとAnthropic Console側で別途予算アラート・spend limitの 設定が必要（README.md「10. 利用料アラート」参照）。

■ AWS IAM（最小権限方針）

各Lambda関数には、その関数が実際に必要とするDynamoDBテーブル・S3バケット・SSMパラメータへのアクセス権限だけをSAMのPolicies（DynamoDBCrudPolicy・S3WritePolicy・SSMParameterReadPolicy等）で個別に付与している。例えばgetSpotsHandlerはSpotsテーブルの読み取りしかできず、書き込みや他テーブルへのアクセス権限は持たない。

4. 主要フローのサービス連携トレース

① ログイン

ブラウザ「ログイン」クリック
→ Cognito Hosted UI

- Google認証画面（Cognitoが仲介）
- Cognitoに戻りIDトークン（JWT）発行
- フロントがJWTを保持し、以後のAPIリクエストにAuthorizationヘッダーとして付与

② 「AI分析を実行」 ボタン

- フロント → API Gateway → adminRunAiBatch（Lambda、UsageTableでレート制限チェック）
- generateSpotScoreBatch を非同期起動して即202を返す
 - generateSpotScoreBatch が裏側で実行：
 - SpotsTable全件取得
 - discover_spots.run_discovery()で新規スポット探索（Google Places API、SSMからキー取得）
 - 各スポットの天気・潮汐取得（Open-Meteo、APIキー不要）
 - スコア計算（ルールベース）
 - スコアが大きく動いたスポットのみClaude APIで理由文生成（SSMからキー取得）
 - RecommendationsTableへ保存
 - フロントは GET /recommendations を数秒おきにポーリングして完了を検知

③ AI相談チャット（画像添付あり）

- フロント → API Gateway（CognitoAuthorizerでJWT検証） → postChatHandler
- UsageTableでレート制限チェック（1日30件）
 - S3(Uploads)から画像バイトを取得
 - SpotsTable・RecommendationsTableをscanして実データのコンテキストを構築
 - Claude APIへテキスト+画像+実データコンテキストを送信（SSMからキー取得）
 - 応答をChatsTableに保存し、同期的にフロントへ返却

④ 写真アップロード（釣果投稿・スポット写真）

- フロント → presignUploadHandler（Lambda）が署名付きPOSTフォームを発行
- ブラウザがS3(Uploads)へ直接POST（Lambda/API Gatewayを経由しない）
 - アップロード完了後、公開URLをPosts/SpotsテーブルにLambda経由で保存

5. なぜこの構成にしたか（設計方針まとめ）

- **サーバーレス徹底:** 個人利用規模のトラフィックに対して常時稼働のサーバーは過剰。使った分だけ課金されるLambda・DynamoDB・S3（+CloudFront）で構成し、待機コストをゼロに近づけている
- **バッチ中心・リアルタイム処理の排除:** スコア計算はEventBridgeの日次バッチと手動実行の2経路に絞り、常時ポーリングするような設計を避けている

- **実データとAI生成の分離:** 座標・実在性など「間違えられないデータ」はGoogle Places API のみを情報源とし、AI (Claude) は説明文生成や魚種推測など「間違っても実害が小さい付随情報」にのみ使う (ハルシネーション対策)
- **最小権限のIAM:** Lambdaごとに必要なリソースへのアクセス権限だけを絞って付与
- **コスト保護の多層化:** ①ユーザー単位の1日あたりレート制限 (UsageTable)、②AWS Budgets (AWS利用料)、③Google Cloud・Anthropic Console側の予算アラート (AWS外の課金)、という3層でコスト暴走を防ぐ設計にしている